Object method, "this"

02 November 2026 18:30

Objects are usually created to represent entities of the real word, like users, orders and so on.

```
let user = {
    name: "John",
    age: 30
};
```

And in the real world, a user can act: select something from the shopping cart, login, logout etc.

Actions are represented in Javascript by functions in properties.

```
let user = {
    name: "John",
    age: 30
};
user.sayHi = function(){
    console.log("Hello!")
};
user.sayHi(); //Hello!
```

Here we have just used a function expression to create a function and assign it to property **user.sayHi** of the object.

Then we can call it as user.sayHi() The user can now speak!

A function that is a property of an object is called its method.

So here we have got a method sayHi of the object user.

Of course we could use a pre - declared function as a method like this.

```
let user = {
    name: "John",
    age: 30
};
```

```
// first declare
function sayHi(){
    console.log("Hello!");
}
// then add as a method
user.sayHi = sayHi;
user.sayHi(); //Hello!
```

Object-oriented Programming
When we write our code using objects to represent entities, that's called Object - Oriented Programming. In short 'OOP'.

Method shorthand
There exist a shorter syntax for method in an object literal:

```
user = {
    sayHi: function(){
        console.log("Hello!");
    }
};
user.sayHi();
```

Alternative syntax:

```
user = {
    sayHi(){ //same as sayHi: function(){...}
        console.log("Hello!");
    }
};
user.sayHi();
```

As demonstrated, we can omit "function" and just write, sayHi()

To tell the truth, the notations are fully identical. There are subtle differences related to object inheritance. But for now, they do not matter. In almost all cases, the shorter syntax is preferred.

"this" in methods
It's common that an object method needs to access the information stored in the object to do its job.

For instance, the code inside user.sayHi() may need the name of the user.

To access the object a method can use the "this" keyword.
The value of this is the object "before dot", the one used to call the method.

For instance:

```
user = {
    name: "John",
    age: 30,
    sayHi(){ //same as sayHi: function(){...}
        console.log("Hello!", this.name);
    }
};
user.sayHi();
```

Output: Hello! John

Here during the execution of user.sayHi(), the value of **this** will be **user.**

Technically, it's also possible to access the object without this by referencing it via the outer variable:

```
user = {
    name: "John",
    age: 30,
    sayHi(){ //same as sayHi: function(){...}
        // console.log("Hello!", this.name);
        console.log("Hello!", user.name);
    }
};
user.sayHi();
```

… But such code is unreliable. If we decide to copy user to another variable. Example admin equal to user and overwrite user with something else then it will access the wrong object.

```
user = {
    name: "John",
    age: 30,
    sayHi(){ //same as sayHi: function(){...}
        // console.log("Hello!", this.name);
        console.log("Hello!", user.name);
    }
};
let admin = user;
user = null; // Overwrite to make things obvious
user.sayHi(); // TypeError: Cannot read property 'name' of null.
```

Output:

```
user.sayHi(); // TypeError: Cannot read property 'name' of null.
     ^

TypeError: Cannot read properties of null (reading 'sayHi')
```

If we used this.name instead of user.name inside the console.log then the code would work.

```
user = {
    name: "John",
    age: 30,
    sayHi(){ //same as sayHi: function(){...}
        // console.log("Hello!", this.name);
        console.log("Hello!", this.name);
    }
};
let admin = user;
user = null; // Overwrite to make things obvious
admin.sayHi();
```

Output:

```
Hello! John
```

"this" is not bound

In Javascript keyword this behaves unlike most other programming languages. It can be used in any function, even if it is not a method of an object.

There's no syntax error in the following example:

```
function sayHi(){
    console.log(this.name);
}
```

The value of this is evaluated during the run - time depending on the context.
For instance, here the same function is assigned to two different objects and has different **"this"** in the calls:

```
let user = { name: "John"};
let admin = { name: "Admin"};
function sayHi(){
    console.log("Hello!", this.name);
}
user.f = sayHi;
admin.f = sayHi;
user.f(); //Hello! John (this == user)
admin.f(); // Hello! Admin (this == admin)
admin['f'](); // Admin (dot or square brackets access the method - doesn't matter)
```

Output:

```
Hello! John
Hello! Admin
Hello! Admin
```

The rule is simple: if obj.f() is called, then **this** is **obj** during the call of **f**. So it's either **user** or **admin** in the example above.

Arrow functions have no "this"
Arrow functions are special: they don't have their own this. If we reference this from such a function, it's taken from the outer "normal" function.
For instance here arrow () uses this from the outer user.sayHi() method.

```
let user = {
    firstName: "John",
    sayHi(){
```

```
            let arrow = () => console.log(this.firstName);
            arrow();
        }
    };
    user.sayHi(); // John
```

Output:
   John

That's a special feature of arrow function. It's useful when we actually don't want to have a separate this, but rather to take it from the outer context.